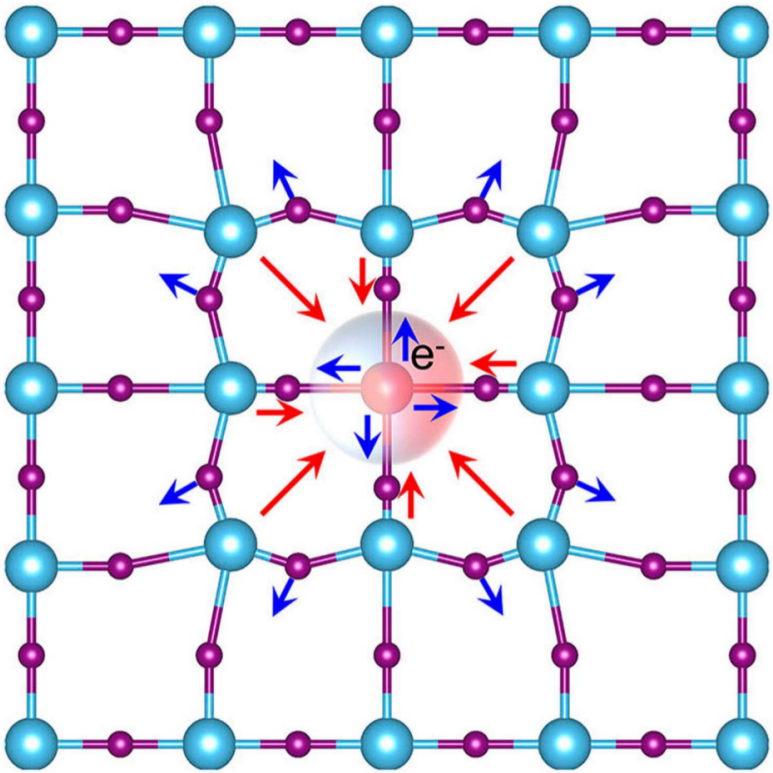
Interactions and Correlations in Condensed Matter

Diagrammatic Monte Carlo - Part I

Cesare Franchini & Thomas Hahn

Table of contents

Review of the polaron problem



- An extra charge (electron or hole) is injected in a lattice
- oppositely-charged ions are *attracted*; equally-charged ions are *shifted away*
- The excess charge is trapped and the lattice distorts locally
- Depending on the spatial extension: small or large polarons
- Prime example of electron-phonon interaction

Numerical and analytical modelling of polarons

First principles (CompMatPhys)

Solution of the many body Schrödinger/Dirac equation

$$\hat{H}\Psi = E\Psi$$

- No empirical assumptions
- No fitting parameters
- Full electronic structure
- Different level of accuracy
- Atomistic interpretation

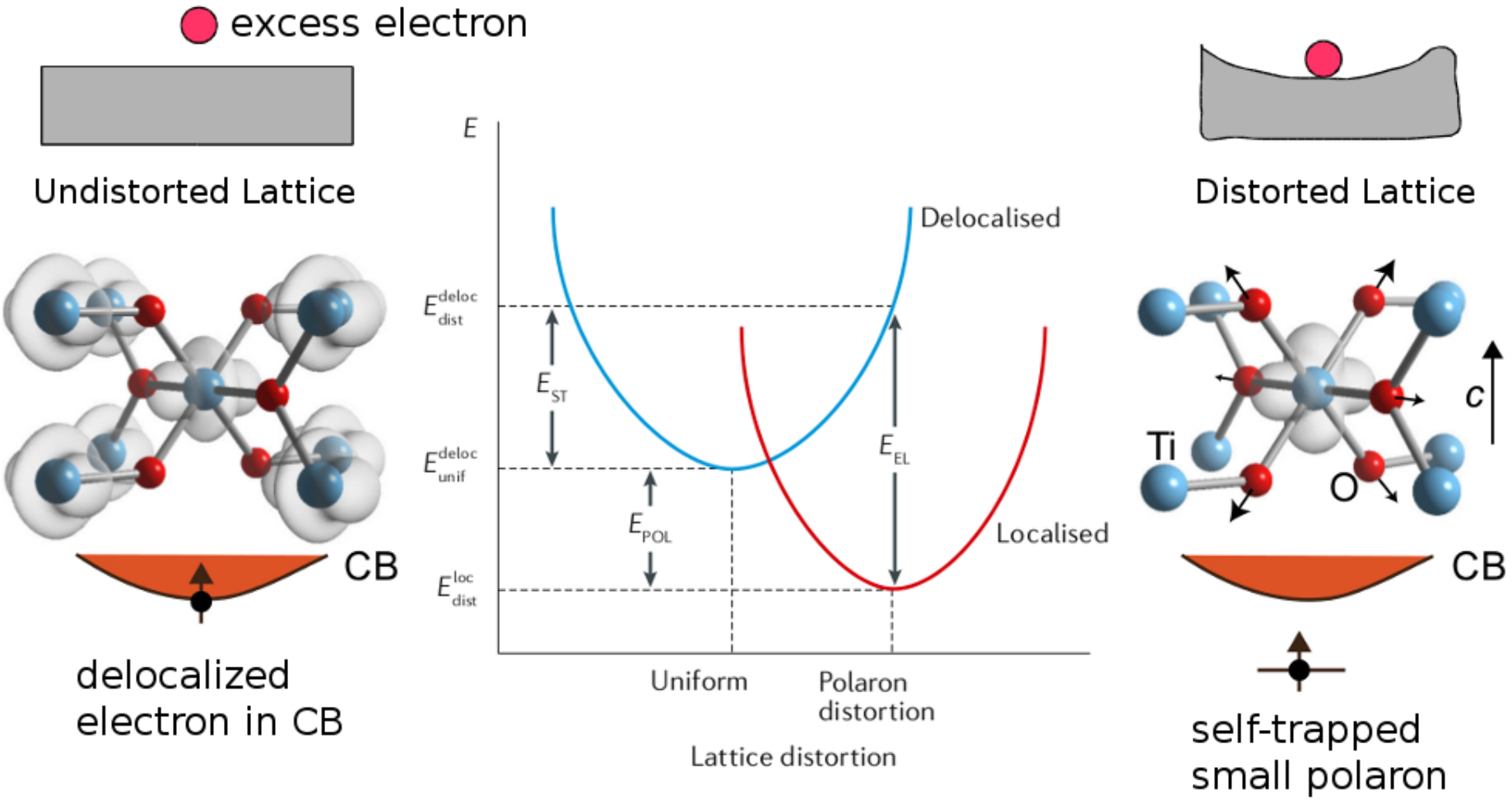
Model Hamiltonian (Int&Corr)

Solution of simplified lattice models (Polaron effective Hamiltonian $\hat{H}_{pol}$)

$$\sum_{\mathbf{k}} \varepsilon_{\mathbf{k}} \hat{a}^{\dagger}_{\mathbf{k}} \hat{a}_{\mathbf{k}} + \sum_{\mathbf{q}} \hbar\omega_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} + \sum_{\mathbf{k},\mathbf{q}} M_{\mathbf{q}} \hat{a}^{\dagger}_{\mathbf{k}+\mathbf{q}} \hat{a}_{\mathbf{k}} \left(\hat{b}_{\mathbf{q}} + \hat{b}^{\dagger}_{-\mathbf{q}} \right)$$

- Restricted Hilbert space (few particles)
- Adjustable parameters
- Many-body correlations
- Accurate solution
- Transparent physical interpretation

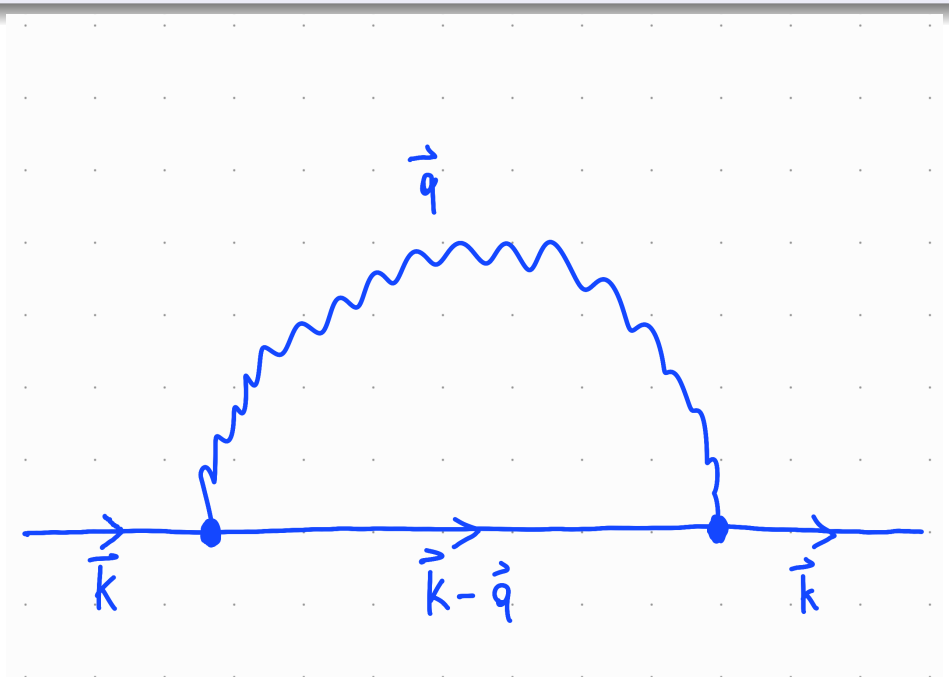
Polarons: First principles



Polaron effective Hamiltonian

Polaron Hamiltonian

$$\hat{H}_{pol} = \sum_{\mathbf{k}} \varepsilon_{\mathbf{k}} \hat{a}^{\dagger}_{\mathbf{k}} \hat{a}_{\mathbf{k}} + \sum_{\mathbf{q}} \hbar\omega_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} + \sum_{\mathbf{k},\mathbf{q}} M_{\mathbf{q}} \hat{a}^{\dagger}_{\mathbf{k}+\mathbf{q}} \hat{a}_{\mathbf{k}} \left(\hat{b}_{\mathbf{q}} + \hat{b}^{\dagger}_{-\mathbf{q}} \right)$$



- electron(s) interacting with phonons
- fully described by $\varepsilon_{\mathbf{k}}$, $\omega_{\mathbf{q}}$ and $M_{\mathbf{q}}$
- different model systems (large vs. small, optical vs. acoustic, etc.)

Polaron Fröhlich Hamiltonian: large polaron

Polaron Hamiltonian

$$\hat{H}_{pol} = \sum_{\mathbf{k}} \varepsilon_{\mathbf{k}} \hat{a}^{\dagger}_{\mathbf{k}} \hat{a}_{\mathbf{k}} + \sum_{\mathbf{q}} \hbar\omega_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} + \sum_{\mathbf{k},\mathbf{q}} M_{\mathbf{q}} \hat{a}^{\dagger}_{\mathbf{k}+\mathbf{q}} \hat{a}_{\mathbf{k}} \left(\hat{b}_{\mathbf{q}} + \hat{b}^{\dagger}_{-\mathbf{q}} \right)$$

Fröhlich polaron:

- large polaron in the continuum approximation
- units: $\hbar = \omega_{LO} = 2m = 1$
- $\varepsilon_{\mathbf{k}} = \frac{\hbar^2 k^2}{2m}$
- $\omega_{\mathbf{q}} = \omega_{LO}$
- $M_{\mathbf{q}} = -i\sqrt{\frac{4\pi\alpha}{V}}\frac{1}{q}\,, \quad \alpha = \frac{e^2}{\hbar}\sqrt{\frac{m}{2\hbar\omega_{LO}}}\left(\frac{1}{\varepsilon_\infty} - \frac{1}{\varepsilon_0}\right)$

Polaron Fröhlich Hamiltonian: large polaron

Polaron Hamiltonian

$$\hat{H}_{pol} = \sum_{\mathbf{k}} \varepsilon_{\mathbf{k}} \hat{a}^{\dagger}_{\mathbf{k}} \hat{a}_{\mathbf{k}} + \sum_{\mathbf{q}} \hbar\omega_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} + \sum_{\mathbf{k},\mathbf{q}} M_{\mathbf{q}} \hat{a}^{\dagger}_{\mathbf{k}+\mathbf{q}} \hat{a}_{\mathbf{k}} \left(\hat{b}_{\mathbf{q}} + \hat{b}^{\dagger}_{-\mathbf{q}} \right)$$

Fröhlich polaron:

- large polaron in the continuum approximation
- units: $\hbar = \omega_{LO} = 2m = 1$
- $\varepsilon_{\mathbf{k}} = \frac{\hbar^2 k^2}{2m}$
- $\omega_{\mathbf{q}} = \omega_{LO}$
- $M_{\mathbf{q}} = -i\sqrt{\frac{4\pi\alpha}{V}}\frac{1}{q}\,, \quad \alpha = \frac{e^2}{\hbar}\sqrt{\frac{m}{2\hbar\omega_{LO}}}\left(\frac{1}{\varepsilon_\infty} - \frac{1}{\varepsilon_0}\right)$

Analytic treatment of Fröhlich polaron:

1. weak-coupling regime, i.e. $\alpha \ll 1 \longrightarrow$ **perturbation theory**
2. strong-coupling regime, i.e. $\alpha \gg 1 \longrightarrow$ **variational approach**

Analytic treatment of Fröhlich polaron

Polaron Hamiltonian

$$\hat{H}_{pol} = \sum_{\mathbf{k}} \varepsilon_{\mathbf{k}} \hat{a}^{\dagger}_{\mathbf{k}} \hat{a}_{\mathbf{k}} + \sum_{\mathbf{q}} \hbar\omega_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} + \sum_{\mathbf{k},\mathbf{q}} M_{\mathbf{q}} \hat{a}^{\dagger}_{\mathbf{k}+\mathbf{q}} \hat{a}_{\mathbf{k}} \left(\hat{b}_{\mathbf{q}} + \hat{b}^{\dagger}_{-\mathbf{q}} \right)$$

- weak-coupling approach $\Longrightarrow$ perturbation theory

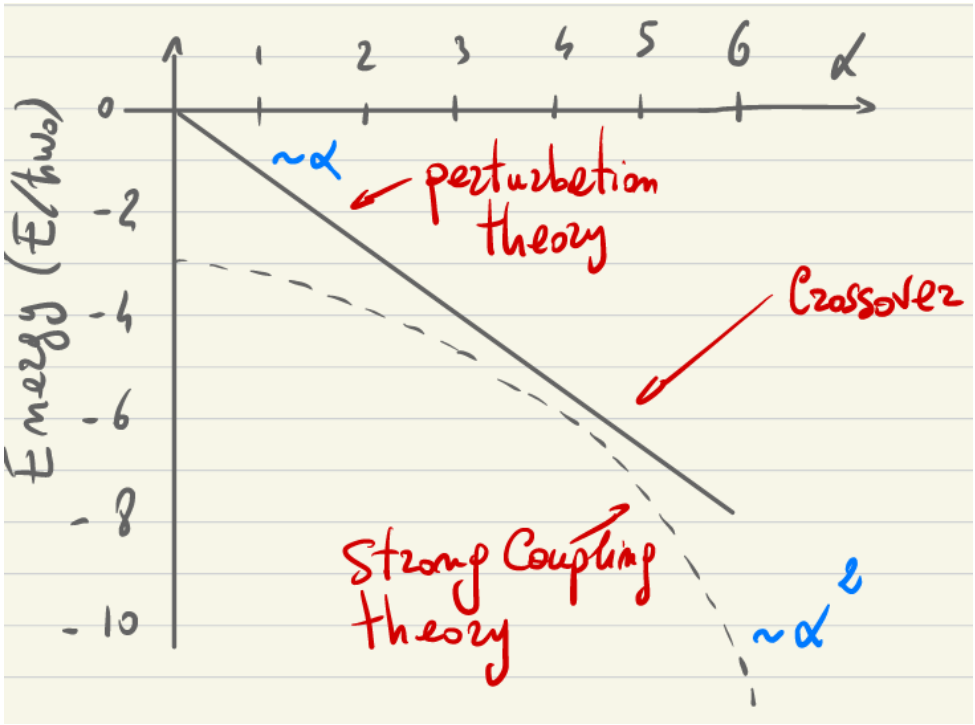
$$E_{\mathbf{k}} = E^{(0)}_{\mathbf{k},\{\emptyset\}} + E^{(2)}_{\mathbf{k},\{\emptyset\}} = -\alpha + k^2 \left(1 - \frac{\alpha}{6}\right)$$

$$m_{pol} = \frac{1}{1 - \frac{\alpha}{6}} \,, \quad \langle N_{ph} \rangle = \langle \mathrm{GS} | \sum_{\mathbf{q}} \hat{b}^{\dagger}_{\mathbf{q}} \hat{b}_{\mathbf{q}} | \mathrm{GS} \rangle \approx \frac{\alpha}{2}$$

- strong-coupling approach $\longrightarrow$ variational treatment

$$r_p = \frac{3}{\alpha} \sqrt{\frac{\pi}{2}} \,, \quad E_{var} = -\frac{\alpha^2}{3\pi}$$

Analytic treatment of Fröhlich polaron



- How good are our simple perturbational and variational estimates?
- What happens in the intermediate coupling regime?
- **How can we improve this result?** $\Longrightarrow$ Diagrammatic Monte Carlo

What is Monte Carlo?

Before jumping right to the Diagrammatic Monte Carlo, we need to review what Monte Carlo methods are and why they are useful in the first place.

What are MC methods?

- huge class of computational algorithms
- common feature $\longrightarrow$ use of random numbers (pseudo RNGs on computers)

What is Monte Carlo?

Before jumping right to the Diagrammatic Monte Carlo, we need to review what Monte Carlo methods are and why they are useful in the first place.

What are MC methods?

- huge class of computational algorithms
- common feature $\longrightarrow$ use of random numbers (pseudo RNGs on computers)

Why are MC methods useful?

- many problems are too complex to be solved with deterministic methods
- only an estimate of the solution is needed
- main applications:
  - evaluating high-dimensional integrals (MC integration)
  - sampling from complex probability densities (Markov Chain Monte Carlo)
  - optimization problems

MC methods in physics?

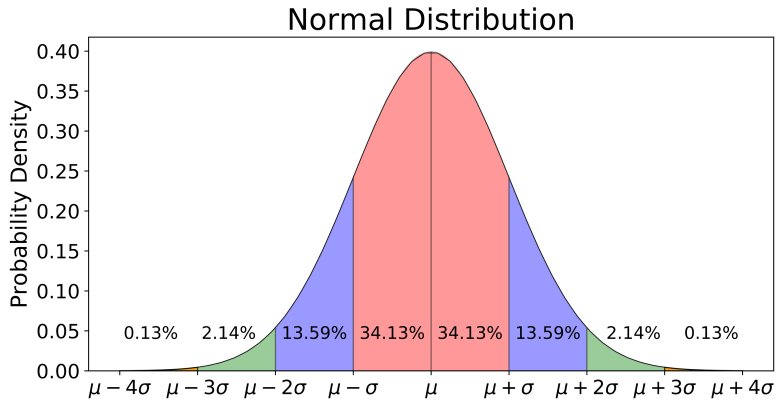
- Classical Monte Carlo
- Quantum Monte Carlo: Variational MC, Diffusion MC, Path integral MC, ...

Review of basic statistics

- Probability density/distribution:

$$p(x) \geq 0$$

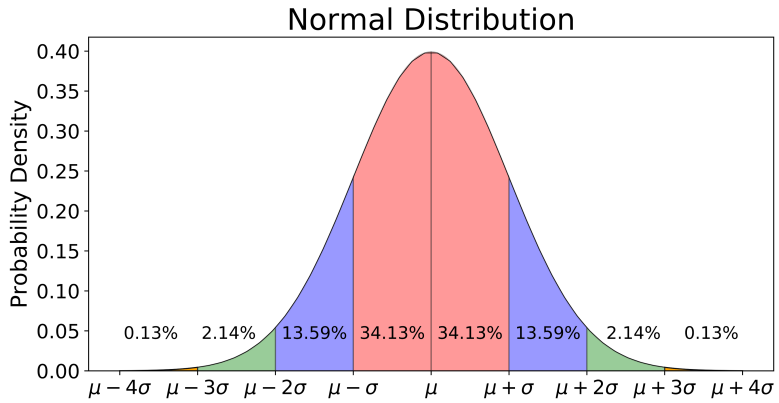
$$\int_\Omega dx\, p(x) = 1$$



Review of basic statistics

- Probability density/distribution:

$$p(x) \geq 0$$
$$\int_\Omega dx\, p(x) = 1$$



- Expectation value: Average value of a quantity over a probability distribution

  Suppose a random variable X is distributed according to $p(x)$

  Expectation value of the random variable X:

$$E[X] = \langle X \rangle_p = \int_\Omega dx\, p(x)x = \sum_i p(x_i)x_i$$

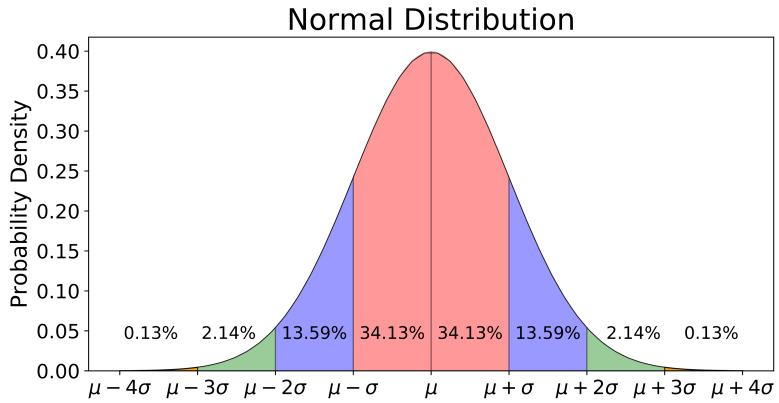
  Expectation value of a function $f(X)$:

$$E[h(X)] = \langle h(X) \rangle_p = \int_\Omega dx\, p(x)h(x) = \sum_i p(x_i)h(x_i)$$

Review of basic statistics

- Probability density/distribution:

$$p(x) \geq 0$$
$$\int_\Omega dx\, p(x) = 1$$



- Expectation value: Average value of a quantity over a probability distribution

  Suppose a random variable X is distributed according to $p(x)$

  Expectation value of the random variable X:

$$E[X] = \langle X \rangle_p = \int_\Omega dx\, p(x)x = \sum_i p(x_i)x_i \approx \frac{1}{N}\sum_{i=1}^{N} x_i = \overline{X}$$

  Expectation value of a function $f(X)$:

$$E[h(X)] = \langle h(X) \rangle_p = \int_\Omega dx\, p(x)h(x) = \sum_i p(x_i)h(x_i) \approx \frac{1}{N}\sum_{i=1}^{N} h(x_i) = \overline{h(X)}$$

  In MC, instead of evaluating the integral or sum, we draw N random samples:

$$x_1, x_2, \ldots, x_N \sim p(x)$$

  This is called the **Monte Carlo estimator**

Monte Carlo estimator: trivial example

$$E[h(X)] \approx \frac{1}{N} \sum_{i=1}^{N} h(x_i) = \overline{h(X)}$$

The key idea is:

expectation value $\approx$ sample average

as long as the samples are drawn from the correct probability distribution.

For example, if x is sampled uniformly from $[0, 1]$, then

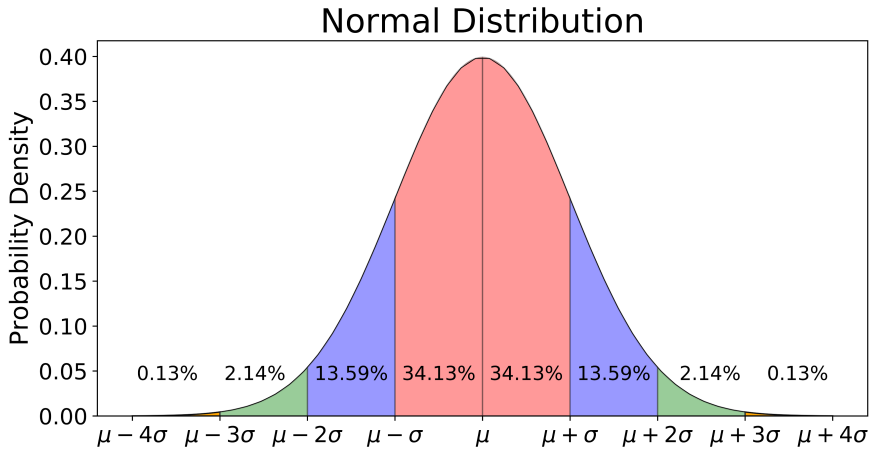
$$\mathbb{E}[x^2] = \int_0^1 x^2 \, dx = \frac{1}{3}$$

MC estimates this by drawing many random numbers $x_k \in [0, 1]$ and computing

$$\mathbb{E}[x^2] \approx \frac{1}{N} \sum_{k=1}^{N} x_k^2$$

As N increases, the estimate approaches the exact value $1/3$.

Back to: Review of basic statistics



- probability density/distribution:

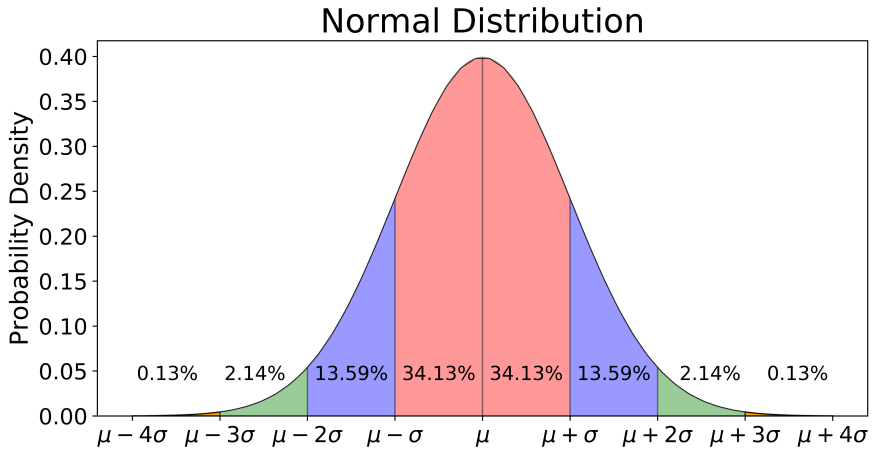
$$p(x) \geq 0$$
$$\int_\Omega dx\, p(x) = 1$$

- expectation value:

$$E[X] = \langle X \rangle_p = \int_\Omega dx\, p(x) x \approx \frac{1}{N} \sum_{i=1}^{N} x_i = \overline{X}$$

$$E[h(X)] = \langle h(X) \rangle_p = \int_\Omega dx\, p(x) h(x) \approx \frac{1}{N} \sum_{i=1}^{N} h(x_i) = \overline{h(X)}$$

Back to: Review of basic statistics



- probability density/distribution:

$$p(x) \geq 0$$
$$\int_\Omega dx\, p(x) = 1$$

- expectation value:

$$E[X] = \langle X \rangle_p = \int_\Omega dx\, p(x) x \approx \frac{1}{N} \sum_{i=1}^{N} x_i = \overline{X}$$

$$E[h(X)] = \langle h(X) \rangle_p = \int_\Omega dx\, p(x) h(x) \approx \frac{1}{N} \sum_{i=1}^{N} h(x_i) = \overline{h(X)}$$

- variance (standard deviation): average squared distance from the mean

$$\sigma_X^2 = E[(X - E[X])^2] \approx \frac{1}{N-1} \sum_{1=1}^{N} (x_i - \overline{X})^2 = s_X^2$$

$$\sigma_{\overline{X}}^2 = \frac{\sigma_X^2}{N} \quad \text{only for uncorrelated samples!}$$

Calculating π with MC

To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

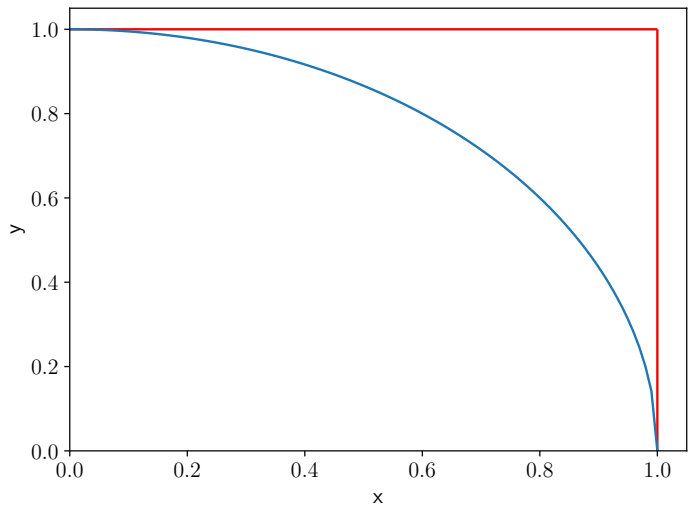
Calculating π with MC

To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

Basic idea:

- Take the quarter unit circle and unit square in the positive quarter plane.

$$A_c = \frac{\pi}{4} \qquad A_s = 1$$



Calculating π with MC

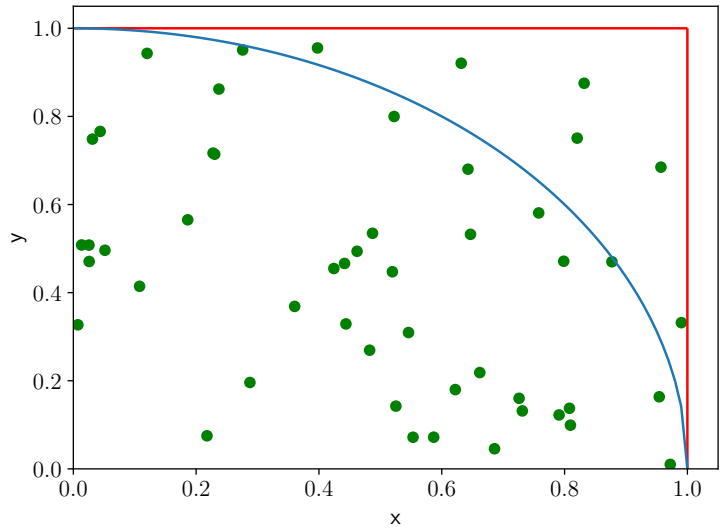
To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

Basic idea:

- Take the quarter unit circle and unit square in the positive quarter plane.

$$A_c = \frac{\pi}{4} \qquad A_s = 1$$

- Generate N random points (x_i, y_i) uniformly distributed in the unit square.



Calculating π with MC

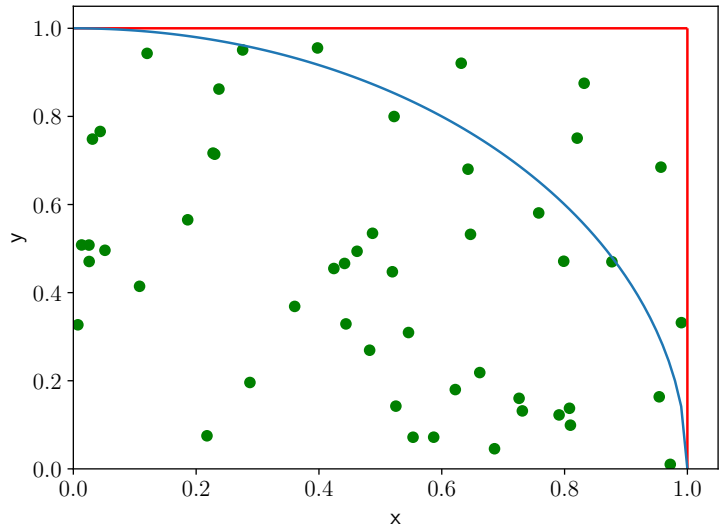
To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

Basic idea:

- Take the quarter unit circle and unit square in the positive quarter plane.

$$A_c = \frac{\pi}{4} \qquad A_s = 1$$

- Generate N random points (x_i, y_i) uniformly distributed in the unit square.
- Count the points N_{in} falling inside the circle.



Calculating π with MC

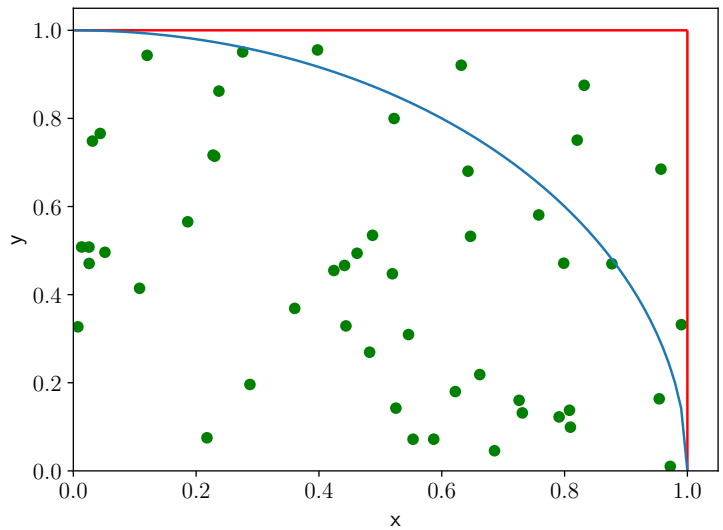
To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

Basic idea:

- Take the quarter unit circle and unit square in the positive quarter plane.

$$A_c = \frac{\pi}{4} \qquad A_s = 1$$

- Generate N random points (x_i, y_i) uniformly distributed in the unit square.
- Count the points N_{in} falling inside the circle.
- Estimate area underneath the circle:
  $A_c \approx \frac{N_{in}}{N}$



Calculating π with MC

To familiarize ourselves with Monte Carlo methods, let us take a look at the famous example of calculating π with MC.

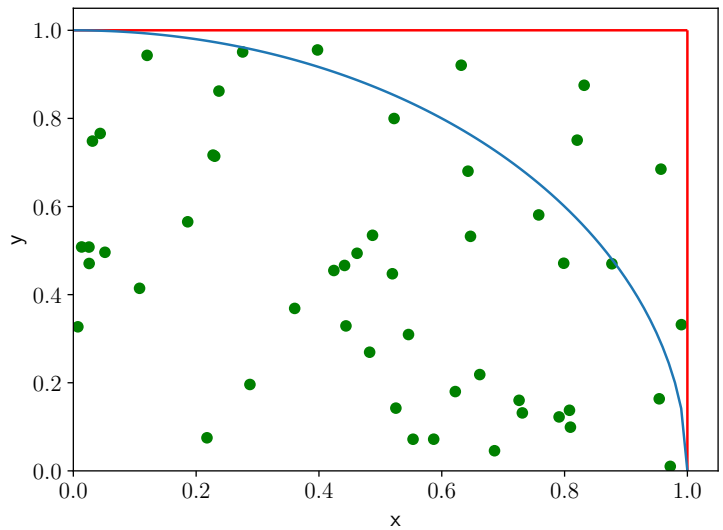
Basic idea:

- Take the quarter unit circle and unit square in the positive quarter plane.

$$A_c = \frac{\pi}{4} \qquad A_s = 1$$

- Generate N random points (x_i, y_i) uniformly distributed in the unit square.
- Count the points N_{in} falling inside the circle.
- Estimate area underneath the circle: $A_c \approx \frac{N_{in}}{N}$

$\Longrightarrow$ Lean back and watch it converge! (`mcpi_dynamic.py`)



Calculating π with MC

Let's take a closer look at what we are actually calculating with this method. Consider the following function (formula of the circle: $x^2 + y^2 = 1$):

$$f(x, y) = \begin{cases} 1 & \text{if} \quad x^2 + y^2 < 1 \\ 0 & \text{otherwise} \end{cases}$$

We can express the area under the circle as a 2D integral over this function:

$$A_c = \frac{\pi}{4} = \int_0^1 \int_0^1 dx\, dy\, f(x, y)$$

Calculating π with MC

Let's take a closer look at what we are actually calculating with this method. Consider the following function (formula of the circle: $x^2 + y^2 = 1$):

$$f(x,y) = \begin{cases} 1 & \text{if} \quad x^2 + y^2 < 1 \\ 0 & \text{otherwise} \end{cases}$$

We can express the area under the circle as a 2D integral over this function:

$$A_c = \frac{\pi}{4} = \int_0^1 \int_0^1 dx\, dy\, f(x,y)$$

Multiplying the integrand by $1 = \frac{p(x,y)}{p(x,y)}$ allows us to rewrite the integral as an expectation value

$$A_c = \int_0^1 \int_0^1 dx\, dy\, \frac{f(x,y)}{p(x,y)} p(x,y) = \left\langle \frac{f(x,y)}{p(x,y)} \right\rangle_p \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i,y_i)}{p(x_i,y_i)}$$

Calculating π with MC

The points are generated uniformly in the unit square, i.e.

$$p(x, y) = \mathcal{U}(0, 1) \times \mathcal{U}(0, 1) = 1$$

Calculating π with MC

The points are generated uniformly in the unit square, i.e.

$$p(x, y) = \mathcal{U}(0, 1) \times \mathcal{U}(0, 1) = 1$$

So we finally get back our original approach

$$A_c = \int_0^1 \int_0^1 dx\, dy\, \frac{f(x, y)}{p(x, y)} p(x, y) \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i, y_i)}{p(x_i, y_i)} = \frac{1}{N} \sum_{i=1}^{N} f(x_i, y_i) = \frac{N_{in}}{N}$$

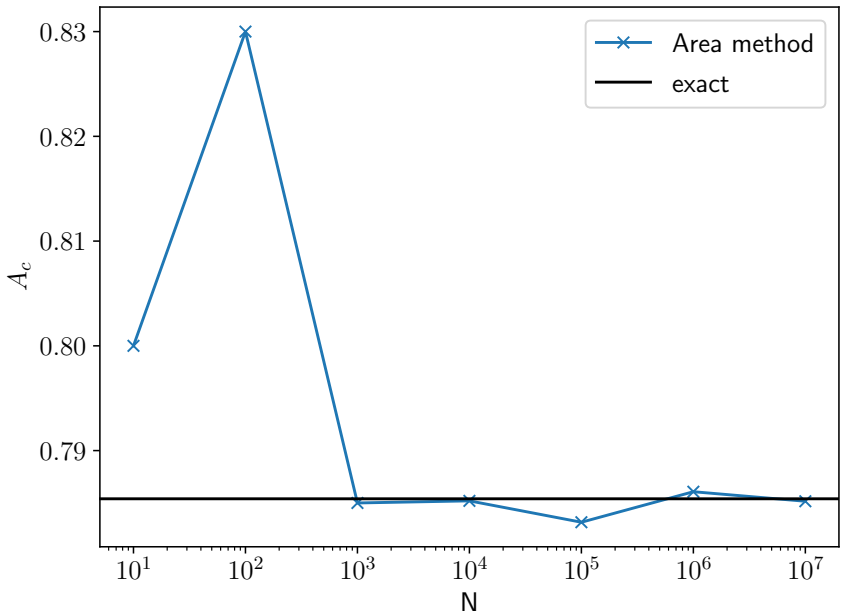
Calculating π with MC

The points are generated uniformly in the unit square, i.e.

$$p(x, y) = \mathcal{U}(0, 1) \times \mathcal{U}(0, 1) = 1$$

So we finally get back our original approach

$$A_c = \int_0^1 \int_0^1 dx\, dy\, \frac{f(x, y)}{p(x, y)} p(x, y) \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i, y_i)}{p(x_i, y_i)} = \frac{1}{N} \sum_{i=1}^{N} f(x_i, y_i) = \frac{N_{in}}{N}$$

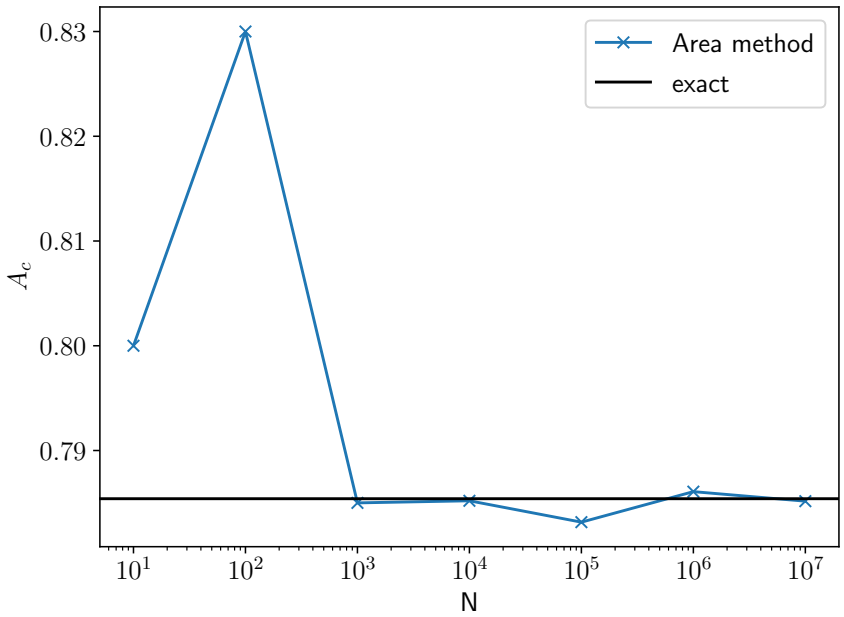


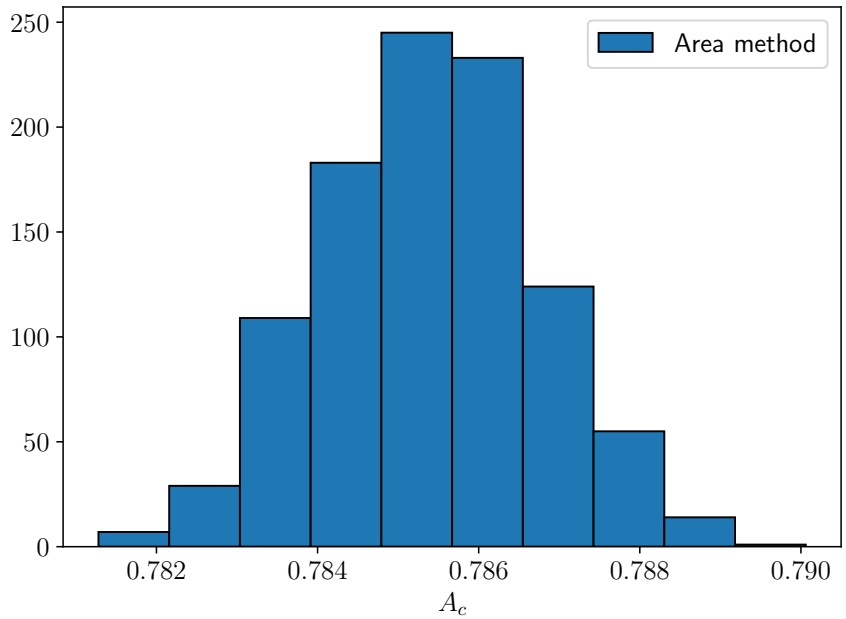
Large number of points N needed for converging to a number close to the exact result.

Low of the large number: if you can conduct large number of trials, than your average converges towards the expected value

Calculating π with MC: The variance of the MC estimators

Generate many estimates of A_c. For each set of estimates, compute the mean (called the Monte Carlo estimate) and the standard deviation (called the Monte Carlo standard error).

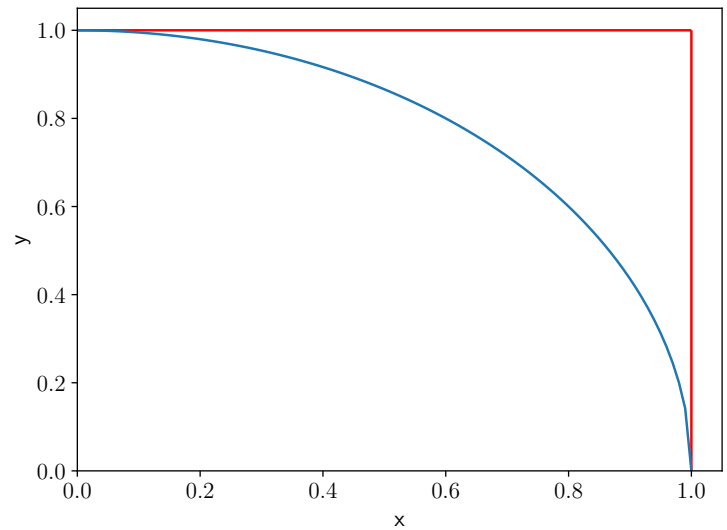




Calculating π with MC

An alternative way is to simply perform the 1-dimension integral

$$A_c = \frac{\pi}{4} = \int_0^1 dx\, f(x) = \int_0^1 dx\, \sqrt{1 - x^2}$$



Calculating π with MC

An alternative way is to simply perform the 1-dimension integral

$$A_c = \frac{\pi}{4} = \int_0^1 dx\, f(x) = \int_0^1 dx\, \sqrt{1-x^2}$$

Again, we can rewrite the integral as an expectation value

$$A_c = \int_0^1 dx\, f(x) = \int_0^1 dx\, \frac{f(x)}{p(x)} p(x) = \left\langle \frac{f(x)}{p(x)} \right\rangle_p \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)}$$

Calculating π with MC

An alternative way is to simply perform the 1-dimension integral

$$A_c = \frac{\pi}{4} = \int_0^1 dx\, f(x) = \int_0^1 dx\, \sqrt{1-x^2}$$

Again, we can rewrite the integral as an expectation value

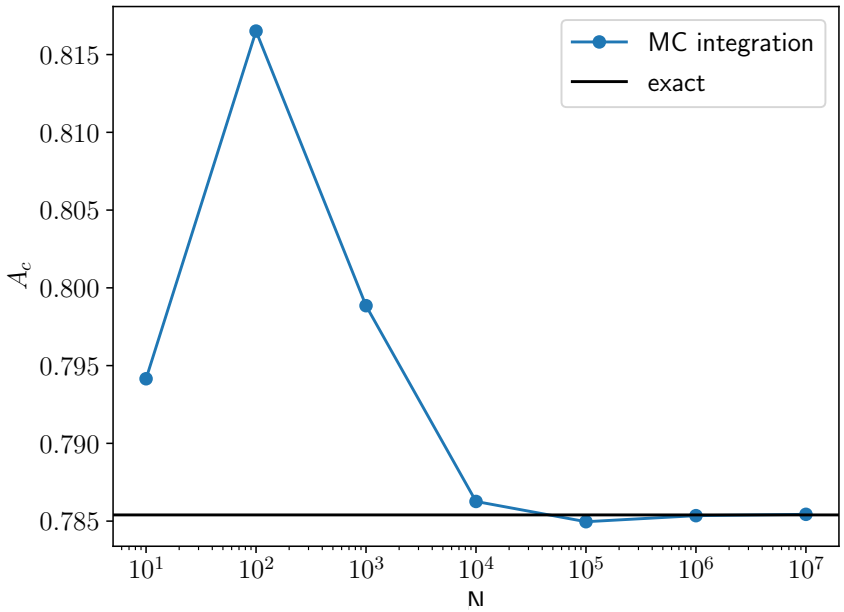
$$A_c = \int_0^1 dx\, f(x) = \int_0^1 dx\, \frac{f(x)}{p(x)} p(x) = \left\langle \frac{f(x)}{p(x)} \right\rangle_p \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)}$$

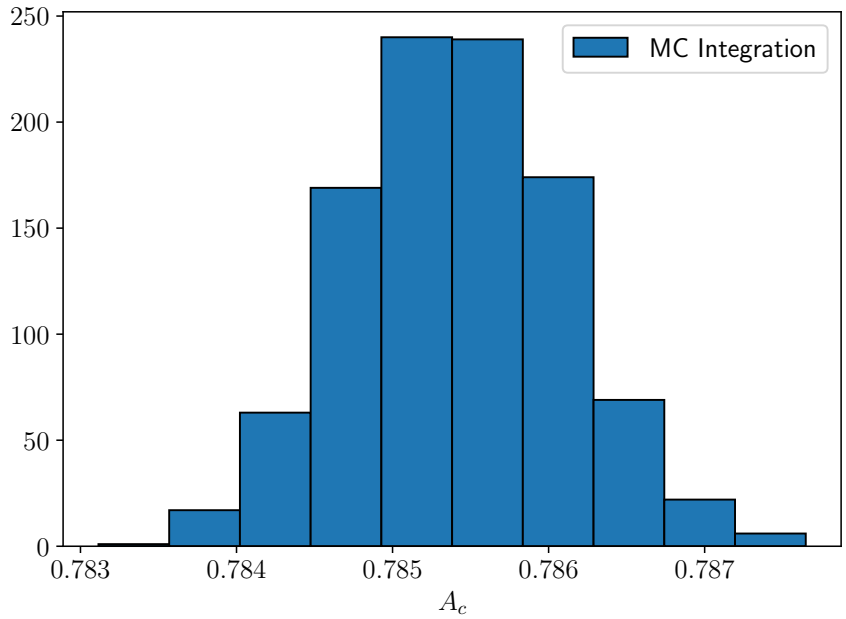
The simplest (*but not best choice*) for $p(x)$ is again a uniform distribution $\mathcal{U}(0,1)$:

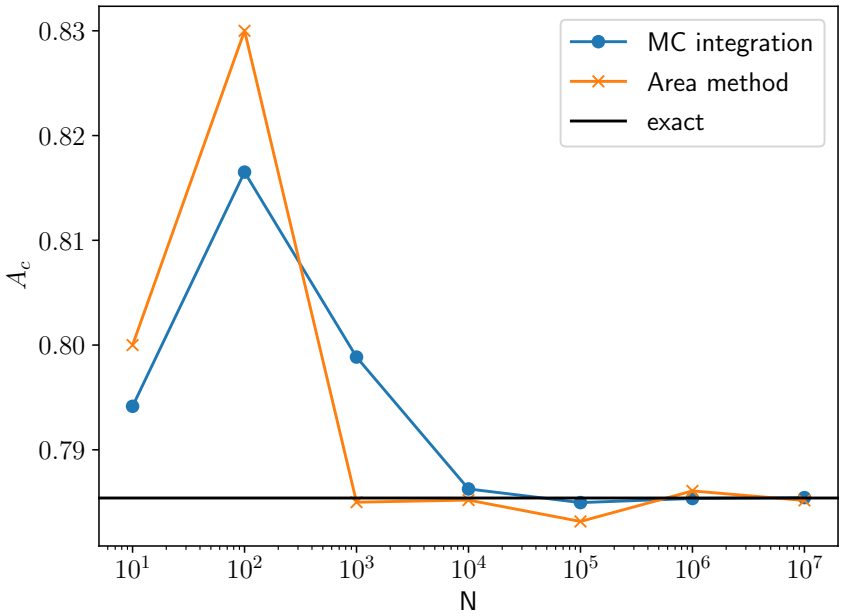
$$A_c = \int_0^1 dx\, \sqrt{1-x^2} \approx \frac{1}{N} \sum_{i=1}^{N} \sqrt{1-x_i^2}$$

where the x_i are drawn from $\mathcal{U}(0,1)$.

Calculating π with MC







Lab Day 1: Exercise 1

Calculating π with MC

Sampling: Why the Uniform distribution is not the best?

In MC integration, one often samples points uniformly in the interval $[0, 1]$.

$$A_c \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)} \approx \frac{1}{N} \sum_{i=1}^{N} \sqrt{1 - x_i^2}, \qquad f(x) = \sqrt{1 - x^2}$$

However, uniform sampling is not always optimal. The function $f(x)$ is larger near $x = 0$ and smaller near $x = 1$. Importance sampling improves the estimate by sampling more frequently in regions where the integrand is large.

Sampling: Why the Uniform distribution is not the best?

In MC integration, one often samples points uniformly in the interval $[0, 1]$.

$$A_c \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)} \approx \frac{1}{N} \sum_{i=1}^{N} \sqrt{1 - x_i^2}, \qquad f(x) = \sqrt{1 - x^2}$$

However, uniform sampling is not always optimal. The function $f(x)$ is larger near $x = 0$ and smaller near $x = 1$. Importance sampling improves the estimate by sampling more frequently in regions where the integrand is large.

The best possible sampling distribution is proportional to the integrand itself:

$$p(x) \propto f(x) = \sqrt{1 - x^2} \quad \overset{\text{normalization}}{=} \quad \frac{4}{\pi} \sqrt{1 - x^2}$$

Sampling: Why the Uniform distribution is not the best?

In MC integration, one often samples points uniformly in the interval $[0, 1]$.

$$A_c \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)} \approx \frac{1}{N} \sum_{i=1}^{N} \sqrt{1 - x_i^2}, \qquad f(x) = \sqrt{1 - x^2}$$

However, uniform sampling is not always optimal. The function $f(x)$ is larger near $x = 0$ and smaller near $x = 1$. Importance sampling improves the estimate by sampling more frequently in regions where the integrand is large.

The best possible sampling distribution is proportional to the integrand itself:

$$p(x) \propto f(x) = \sqrt{1 - x^2} \quad \overset{\text{normalization}}{=} \quad \frac{4}{\pi} \sqrt{1 - x^2}$$

With this choice,

$$\frac{f(x)}{p(x)} = \frac{\sqrt{1 - x^2}}{\frac{4}{\pi}\sqrt{1 - x^2}} = \frac{\pi}{4}.$$

Every sampled point gives the same value, and the variance is zero!

Sampling: importance-sampling in pratice

In practice, there is a catch with ideal importance-sampling: to normalize $p(x)$, one already needs to know

$$\int_0^1 \sqrt{1-x^2}\, dx = \frac{\pi}{4}. \tag{1}$$

This exact optimal distribution is not useful if the goal is compute the area!

Sampling: importance-sampling in pratice

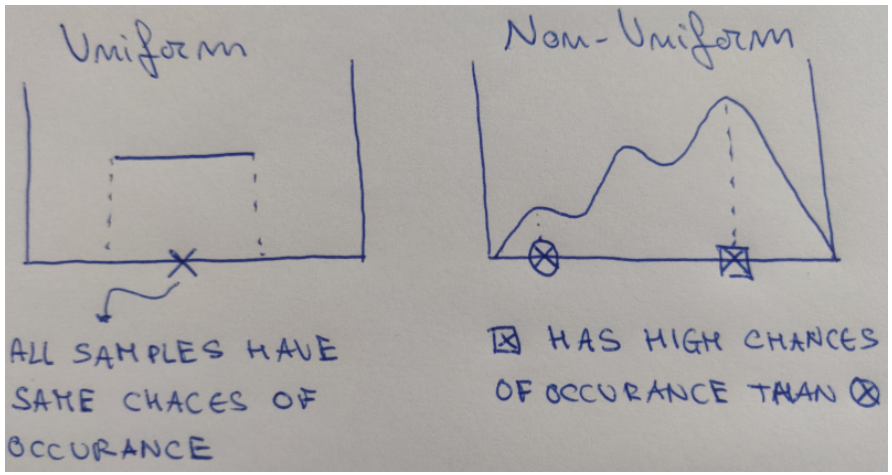
In practice, there is a catch with ideal importance-sampling: to normalize $p(x)$, one already needs to know

$$\int_0^1 \sqrt{1-x^2}\, dx = \frac{\pi}{4}. \tag{1}$$

This exact optimal distribution is not useful if the goal is compute the area!

Practical lesson: importance sampling works best when the sampling distribution has a shape similar to the integrand.

Uniform sampling is simple, but not optimal. A better choice samples more often where the integrand is large and less often where it is small.



What if we cannot sample directly from the distribution we want?

Markov Chain

In many real problems, direct sampling from $p(x)$ is difficult or impossible. Solution: Markov chain!

Markov chain is a process for which predictions can be made regarding future outcomes based solely on its present state. Instead of independent samples, we generate a correlated sequence:

$$x_0 \to x_1 \to x_2 \to x_3 \cdots$$

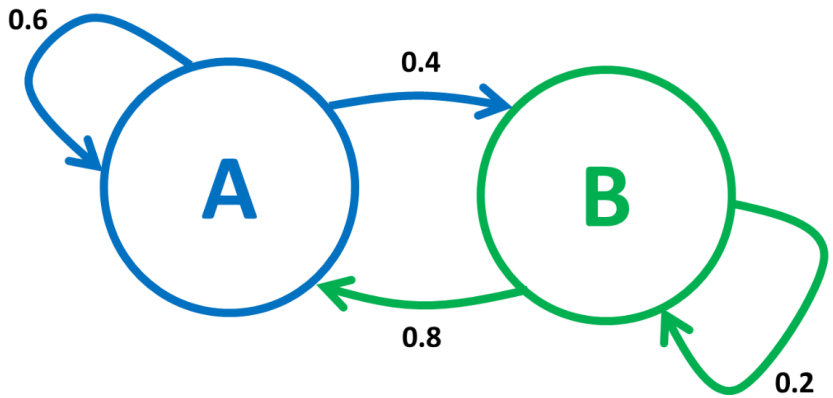
where each new point is proposed from the previous one.

Key idea: We want the chain to spend more time where $p(x)$ is large, and less time where $p(x)$ is small.

Markov Chain

Markov property: A process is Markovian if the next step only depends on the current position.

A two-state Markov process:

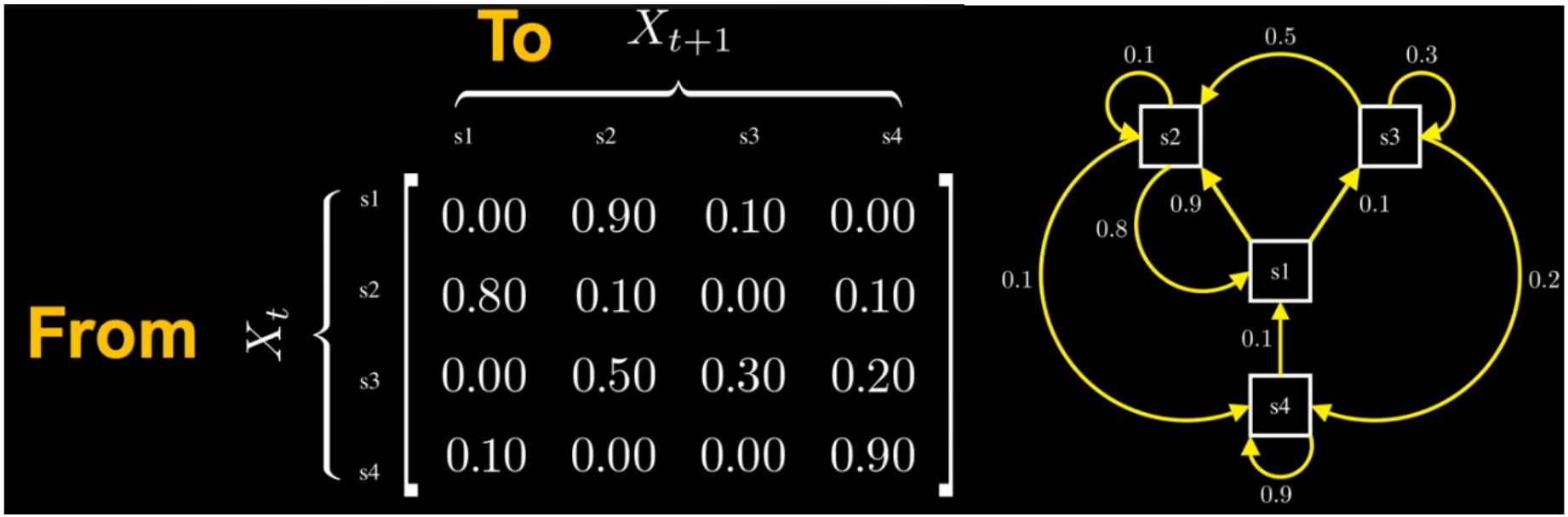


1. Start at the current point A
2. Propose a small random move to a new point B
3. If the new point has higher probability, accept it. (The numbers are the probability of changing from one state to another state.)

Markov Chain: Transition Probability Matrix

Consider a state space $\mathcal{S}$. Suppose the Markov chain has states: s_1, s_2, s_3 and s_4

The transition matrix contains entries p_{ij} = probability of going from state i to state j in one step



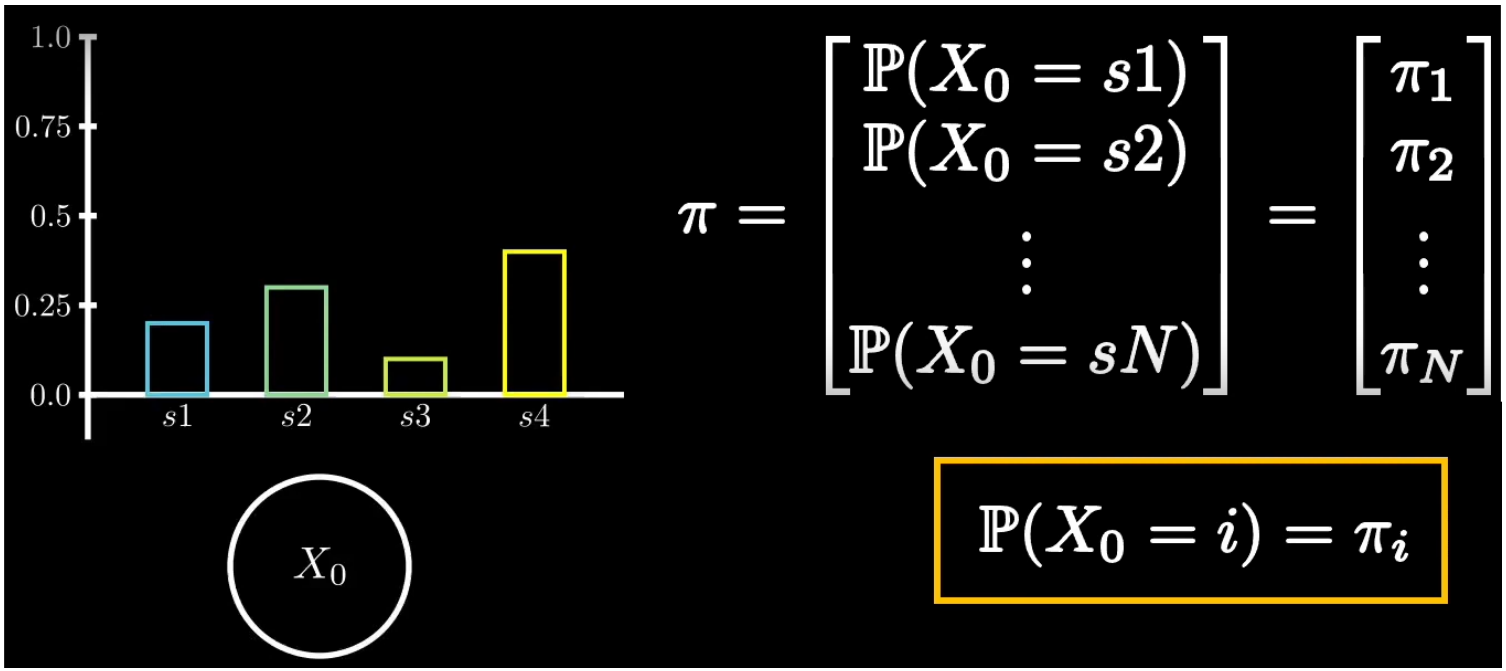
$$p(X_{t+1} = j | X_t = i) = p_{ij} \qquad (\text{or } \omega_{XX'})$$

We have clarified the concept of transition probabilities. The next step is: Finding out the probability distribution of a random variable X at t=1, knowing its probability distribution at t=0.

Markov Chain: Probability distribution

Knowing the probability distribution at t=0, how to find the distribution at t=1?

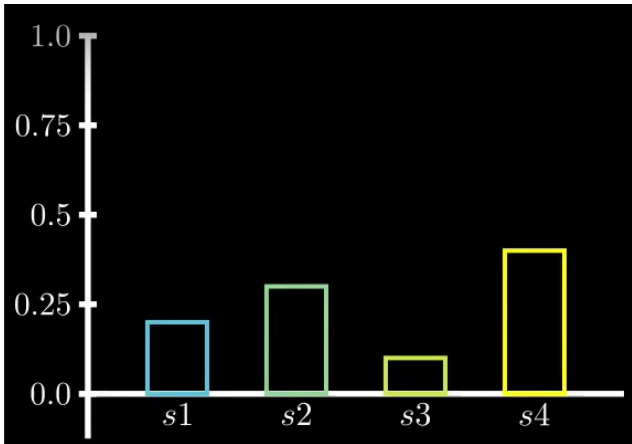
Distribution at t=0: π



Markov Chain: Probability distribution

Knowing the probability distribution at t=0, how to find the distribution at t=1?

Distribution at t=0: π



$$\pi = \begin{bmatrix} \mathbb{P}(X_0 = s1) \\ \mathbb{P}(X_0 = s2) \\ \vdots \\ \mathbb{P}(X_0 = sN) \end{bmatrix} = \begin{bmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_N \end{bmatrix}$$

$$\mathbb{P}(X_0 = i) = \pi_i$$

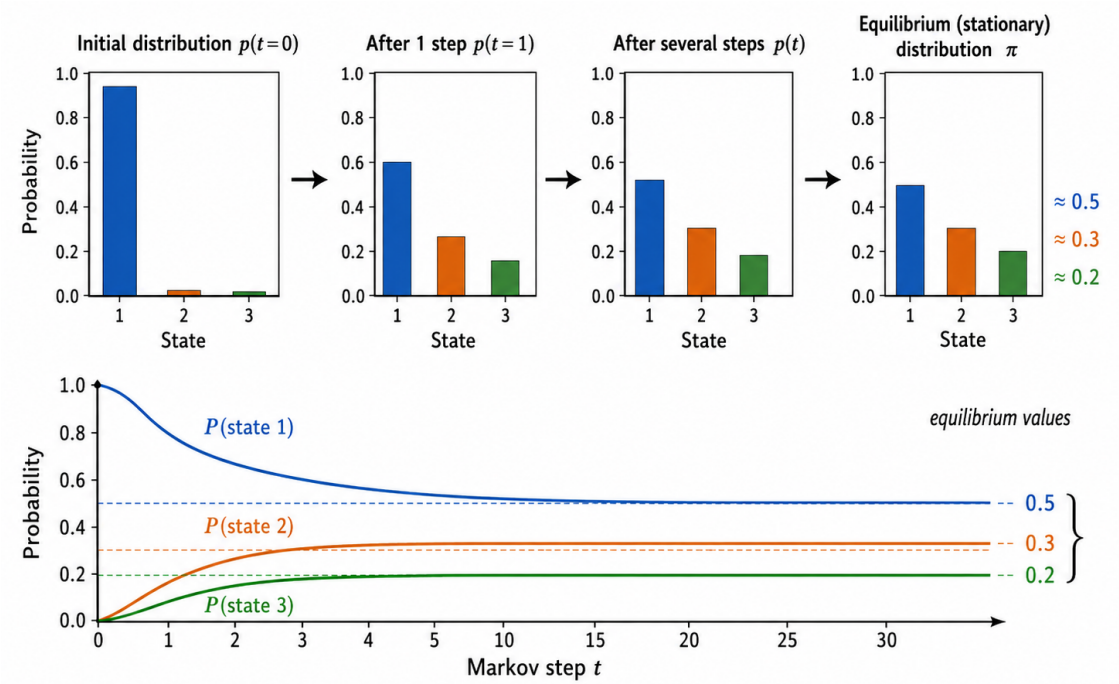
Distribution at t=1:

$$\mathbb{P}(X_1 = j) = \sum_{i=1}^{N} \mathbb{P}(X_i = j | X_0 = i)\mathbb{P}(X_0 = i) = \sum_{i=1}^{N} p_{ij}\pi_i = \sum_{i=1}^{N} \pi_i p_{ij}$$

To get the distribution of a random variable at time step 1 we need to multiply the distribution of random variable at t=0 with the transition matrix.

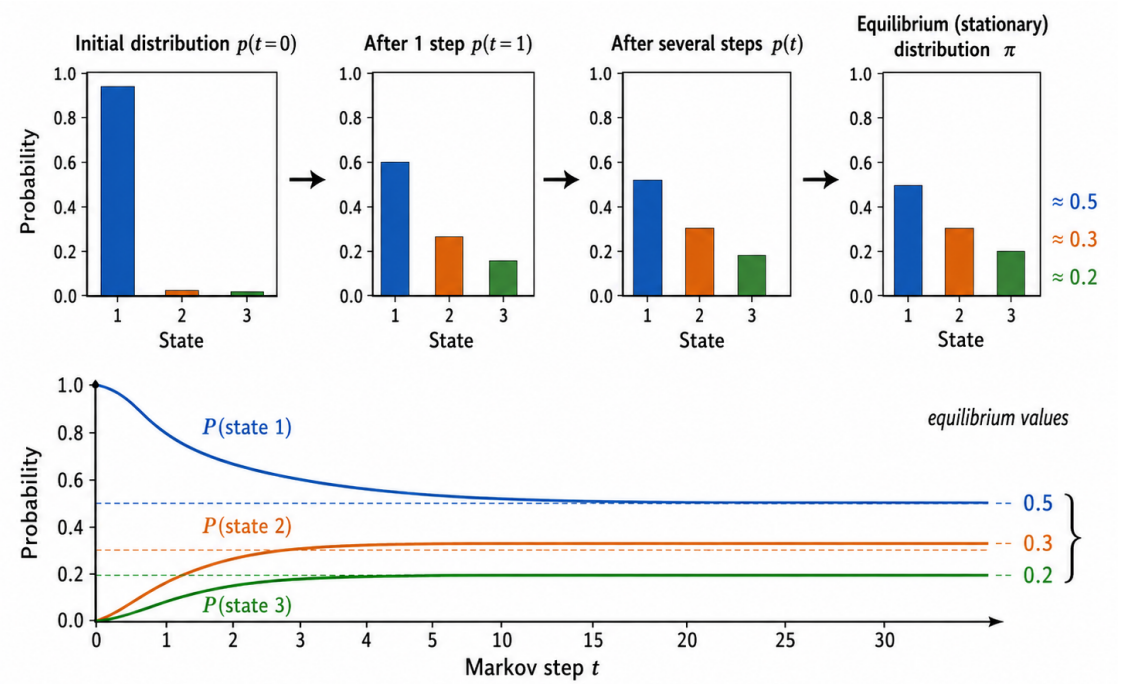
Markov Chain: stationary distribution

After a sufficient number of time steps our distribution will reach the equilibrium distribution: it does not change anymore when the transition matrix is applied.



Markov Chain: stationary distribution

After a sufficient number of time steps our distribution will reach the equilibrium distribution: it does not change anymore when the transition matrix is applied.



Main goal: building a Markov chain whose equilibrium distribution is exactly the desired target distribution:

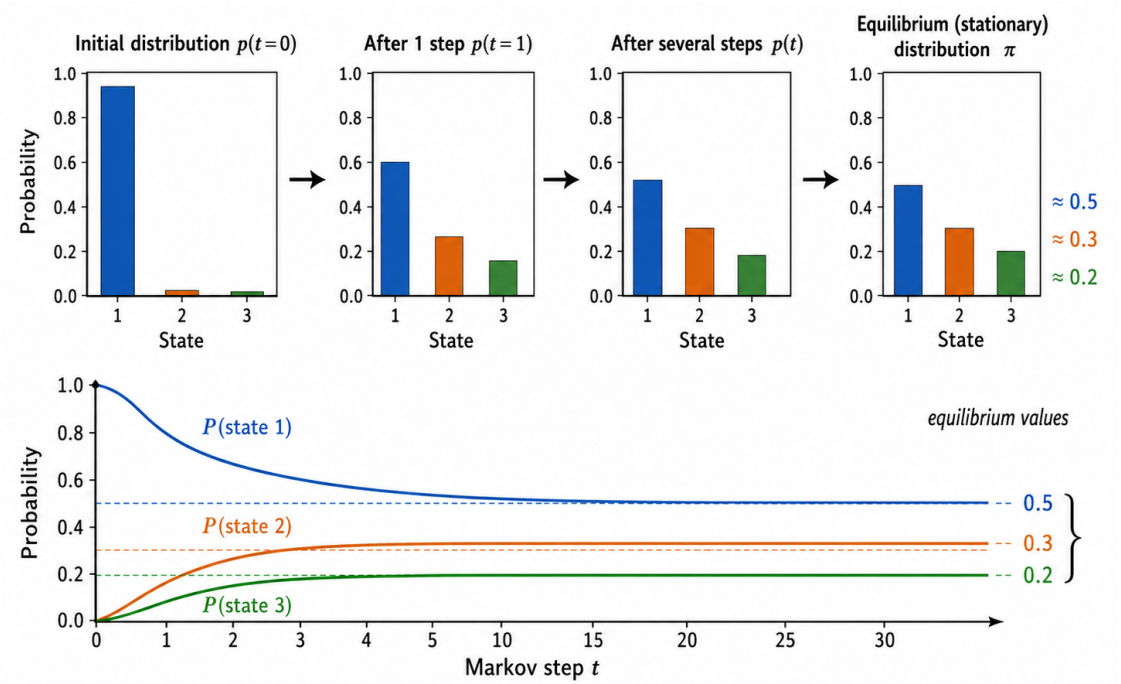
If $p(x)$ large near some values of $x \rightarrow$ MC samples should visit those values often.

If $p(x)$ is small somewhere else $\rightarrow$ samples should visit that region rarely.

A Markov chain that has converged to a stationary distribution will enable you to revisit the various states in accordance with their respective plausibility.

Markov Chain: stationary distribution

After a sufficient number of time steps our distribution will reach the equilibrium distribution: it does not change anymore when the transition matrix is applied.



Main goal: building a Markov chain whose equilibrium distribution is exactly the desired target distribution:

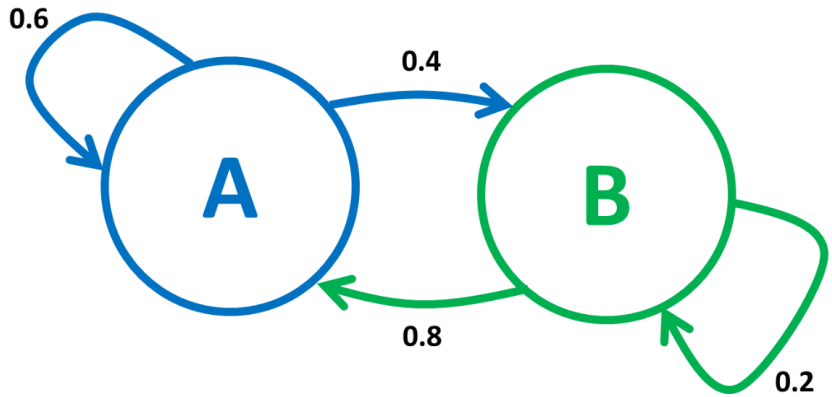
If $p(x)$ large near some values of $x \rightarrow$ MC samples should visit those values often.

If $p(x)$ is small somewhere else $\rightarrow$ samples should visit that region rarely.

A Markov chain that has converged to a stationary distribution will enable you to revisit the various states in accordance with their respective plausibility.

How to create such an artificial Markov chain? Metropolis-Hastings

Markov Chain Monte Carlo (MCMC): Summing up



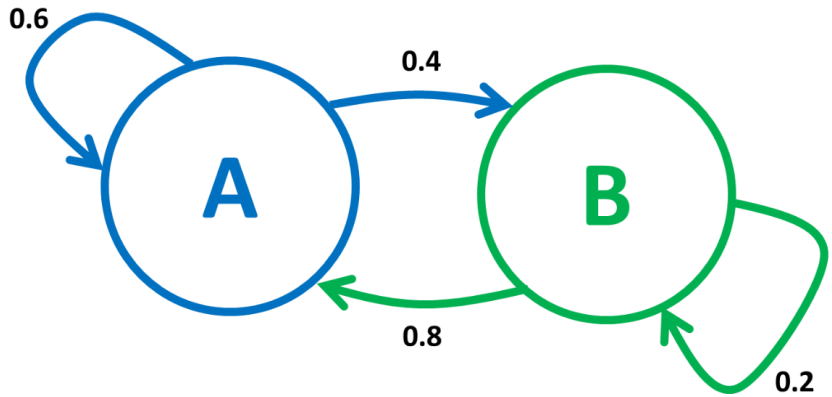
Markov chains:

- state space $\mathcal{S}$ (discrete or continuous)
- transition probabilities $\omega_{XX'}$: probability to go to state X' if the current state is X
- initial distribution $\pi^{(0)}$
- stationary distribution $p(X) = \sum_j \omega_{XX'} p(X')$

$\Longrightarrow$ once the Markov chain is in its stationary distribution, we can draw random samples from $p(X)$

$\Longrightarrow$ How do we choose $\omega_{XX'}$ that we end up in $p(X)$?

Markov Chain Monte Carlo (MCMC): Summing up



Markov chains:

- state space $\mathcal{S}$ (discrete or continuous)
- transition probabilities $\omega_{XX'}$: probability to go to state X' if the current state is X
- initial distribution $\pi^{(0)}$
- stationary distribution $p(X) = \sum_j \omega_{XX'} p(X')$

$\Longrightarrow$ once the Markov chain is in its stationary distribution, we can draw random samples from $p(X)$

$\Longrightarrow$ How do we choose $\omega_{XX'}$ that we end up in $p(X)$?

Markov Chain Monte Carlo:

- algorithms for sampling from complex probability distributions
- generates a Markov chain with the desired distribution as its equilibrium distribution
- most famous algorithm: Metropolis-Hastings algorithm

Metropolis-Hastings algorithm

Metropolis-Hastings algorithm: practical recipe that tells us how to build a Markov chain whose equilibrium distribution is the target distribution.

Key question: How should we choose the transition probabilities so that the chain visits points according to p(x)?

Key point: choosing a clever acceptance/rejection scheme helping us in generating our target distribution.

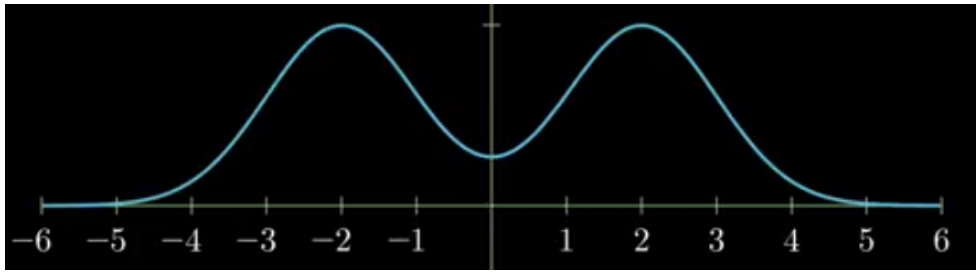
Main steps:

1. Define target distribution function f(x)
2. Start from the first sample x_t
3. Propose a new point x_{t+1}
4. Compare the target probability at the new point $p(x_{t+1})$ with the target probability at the current point $p(x_t)$.
5. Accept or reject the move according to a specific probability.
6. If the move is accepted, the chain moves to x_{t+1}.
7. If the move is rejected, the chain stays at x_t.

Remarkable feature: Metropolis-Hastings does not require knowing the normalization constant of the target distribution!

Metropolis-Hastings algorithm: Simple Example

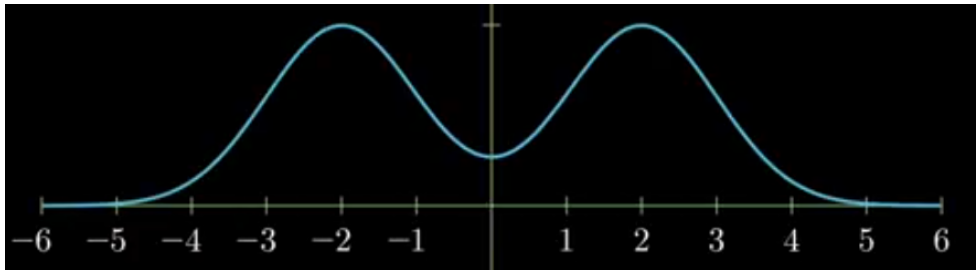
Target distribution: $f(x) = 0.5\mathcal{N}(-2, 1) + 0.5\mathcal{N}(2, 1)$



Mixing of two Gaussian ($\mathcal{N}$) random variables. Our goal is to obtain the samples in accordance with this distribution function.

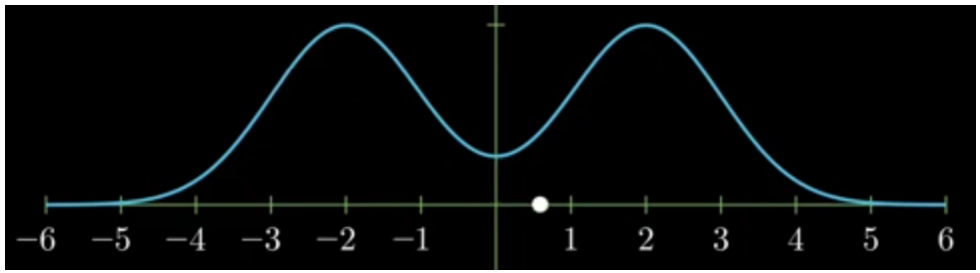
Metropolis-Hastings algorithm: Simple Example

Target distribution: $f(x) = 0.5\mathcal{N}(-2, 1) + 0.5\mathcal{N}(2, 1)$



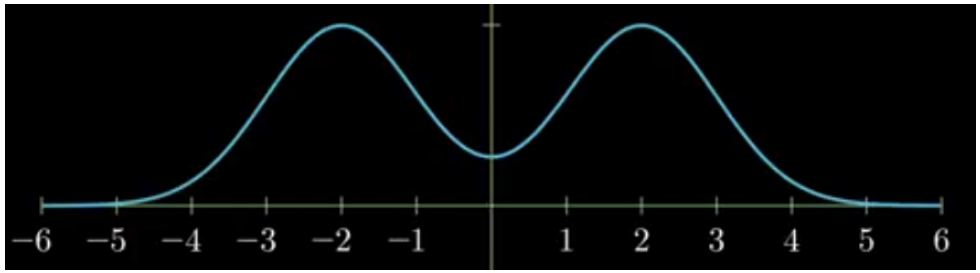
Mixing of two Gaussian ($\mathcal{N}$) random variables. Our goal is to obtain the samples in accordance with this distribution function.

Initial value: $x_t = 0.6$



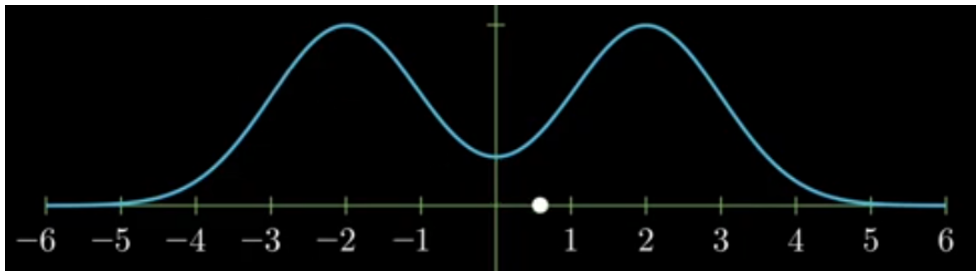
Metropolis-Hastings algorithm: Simple Example

Target distribution: $f(x) = 0.5\mathcal{N}(-2, 1) + 0.5\mathcal{N}(2, 1)$



Mixing of two Gaussian ($\mathcal{N}$) random variables. Our goal is to obtain the samples in accordance with this distribution function.

Initial value: $x_t = 0.6$



Next sample: x_{t+1} ? proposal distribution

Metropolis-Hastings algorithm: Simple Example

We want to make a Markov chain of our sample: adding some random perturbance to our current sample by means of a proposal distribution

Next sample: $x_{t+1} = x_t + \mathcal{N}(0, \sigma^2) \sim \mathcal{N}(x_t, \sigma^2)$

Proposal distribution: rule used to suggest the next possible move, in this case the normal distribution $\mathcal{N}(0, \sigma^2)$

For $x_t = 0.6$ and $\sigma^2 = 0.4$ and $x_{t+1} = 1.2$

Metropolis-Hastings algorithm: Simple Example

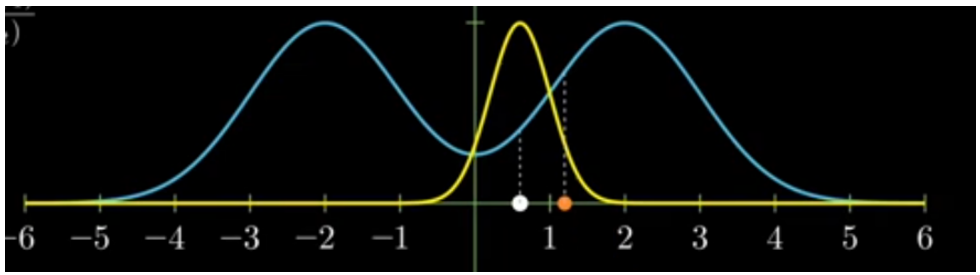
We want to make a Markov chain of our sample: adding some random perturbance to our current sample by means of a proposal distribution

Next sample: $x_{t+1} = x_t + \mathcal{N}(0, \sigma^2) \sim \mathcal{N}(x_t, \sigma^2)$

Proposal distribution: rule used to suggest the next possible move, in this case the normal distribution $\mathcal{N}(0, \sigma^2)$

For $x_t = 0.6$ and $\sigma^2 = 0.4$ and $x_{t+1} = 1.2$

x_{t+1} **accepted or rejected?** We compute x_t and x_{t+1} from our target function



Define acceptance probability $\alpha = min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right)$

Metropolis-Hastings algorithm: Simple Example

$$\alpha = min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right)$$

Acceptance criteria: we use of a random variable $u \sim \mathcal{U}(0,1)$, as often in MC.

if $\alpha > u$ move to proposed sample $x_{t+1} = 1.2$
STORAGE = [0.6, 1.2]

if $\alpha < u$ stay at current sample $x_{t+1} = x_t = 0.6$
STORAGE = [0.6, 0.6]

$\Rightarrow$ See step by step example in virtuale (MCMC bimodal)

Metropolis-Hastings algorithm: Simple Example

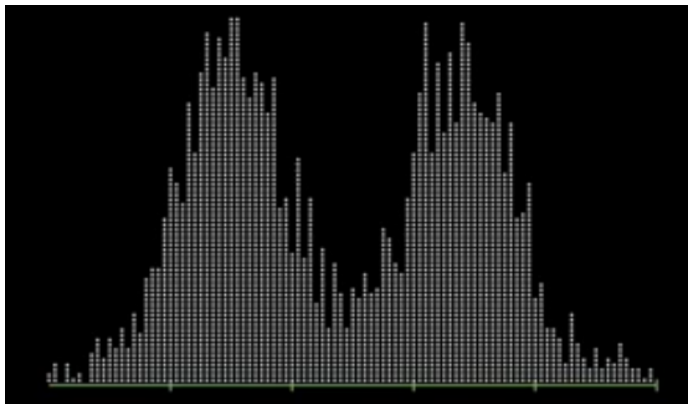
$$\alpha = min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right)$$

Acceptance criteria: we use of a random variable $u \sim \mathcal{U}(0,1)$, as often in MC.

if $\alpha > u$ move to proposed sample $x_{t+1} = 1.2$
STORAGE = [0.6, 1.2]

if $\alpha < u$ stay at current sample $x_{t+1} = x_t = 0.6$
STORAGE = [0.6, 0.6]

Final step: run the algorithm for some time, collect storage and plot corresponding histograms: stationary state $\sim$ target function (same shape).



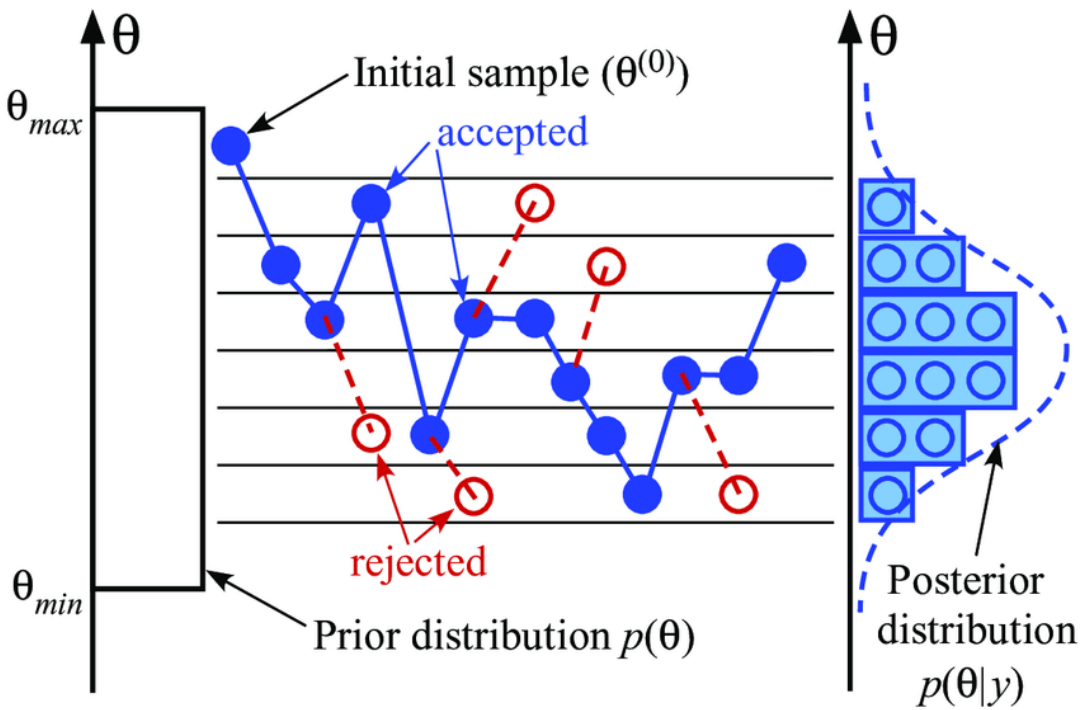
$\Rightarrow$ See step by step example in virtuale (MCMC bimodal)

Metropolis-Hastings algorithm

More general form of the (Metropolis-Hastings) acceptance probability:

$$\alpha = \frac{f(x_{t+1})}{f(x_t)} \frac{q(x_t|x_{t+1})}{q(x_{t+1}|x_t)}$$

Where q is a conditional probability distribution for the new value given the current value. In this form, the acceptance probability does consider the proposal distribution. If the proposal distribution is symmetric this additional term is 1.



Lab Day 1: Exercise 2

Approximating the function $e^{-\alpha\tau}$ with MCMC

Metropolis-Hastings algorithm

Input : initial distribution $\pi^{(0)}$
target distribution $p(X)$
proposal distribution $w_{XX'}$
Output: random samples according to $p(X)$

samples[];
start from initial state $X \sim \pi^{(0)}$;
perform thermalization steps;
while *not enough samples* **do**
  draw a proposal sample X' from $w_{XX'}$;
  calculate $p(X)$ and $p(X')$;
  if $w_{X'X}p(X') \geq w_{XX'}p(X)$ **then**
    $X \leftarrow X'$;
  else
    calculate $R = \frac{w_{X'X}p(X')}{w_{XX'}p(X)}$;
    draw random uniform number r;
    if $r \leq R$ **then**
      $X \leftarrow X'$;
  end
  samples.add(X);
end
return samples;

FIGURE 4.2: Metropolis-Hastings algorithm.

Detailed balance for the Markov chain

Detailed balance: condition that guarantees that a Markov chain has the desired equilibrium distribution: every transition is statistically balanced by its reverse transition.

$$p_{curr}(x)p_{curr}(x'|x) = p_{proposal}(x')p_{rev}(x|x')$$

$p_{curr}(x)$: current probability
$p_{curr}(x'|x)$: current transition probability $x \to x'$
$p_{proposal}(x)$: proposal probability
$p_{rev}(x|x')$: reverse transition probability $x' \to x$

Detailed balance for the Markov chain

Detailed balance: condition that guarantees that a Markov chain has the desired equilibrium distribution: every transition is statistically balanced by its reverse transition.

$$p_{curr}(x)p_{curr}(x'|x) = p_{proposal}(x')p_{rev}(x|x')$$

$p_{curr}(x)$: current probability
$p_{curr}(x'|x)$: current transition probability $x \to x'$
$p_{proposal}(x)$: proposal probability
$p_{rev}(x|x')$: reverse transition probability $x' \to x$

At equilibrium, if $p(x)$ is the target distribution:

$$p(x)p(x'|x) = p(x')p(x|x')$$

Detailed balance means that, at equilibrium, every transition is statistically balanced by its reverse transition. It is important because it gives us a simple test that the Markov chain will sample the correct distribution.

Approximating a distribution with MCMC

Goal: Approximate the function $Q(\tau) = e^{-\alpha\tau}$ for $\alpha = 1$ and $\tau \in [0, 5]$ using MCMC.

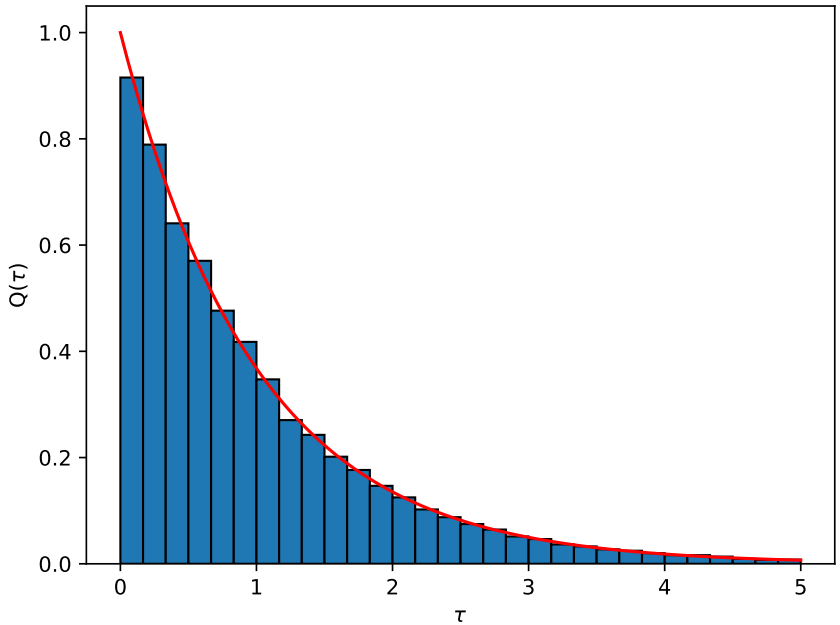
Basic idea:

- interpret $Q(\tau)$ as a distribution and draw samples from it
- put the samples into a histogram $H(\tau)$ to reconstruct (the shape of) $Q(\tau)$
- normalize the histogram properly such that $H(\tau) \approx Q(\tau)$

Implementation details:

1. discrete τ-space into M bins β_i of size $\Delta\tau_i$ and set the bin counters $N_i = 0$
2. start from an arbitrary initial position $\tau \in [0, 5]$
3. propose a new position τ' from $p(\tau'|\tau)$, e.g. $\mathcal{U}(0, 5)$
4. accept the new position with probability $P_{acc} = \min\left\{1, \frac{Q(\tau')p(\tau|\tau')}{Q(\tau)p(\tau'|\tau)}\right\}$, i.e. $\tau \to \tau'$ if accepted
5. increase the bin counter $N_i += 1$ for which $\tau \in \beta_i$
6. go back to 3

Approximating a distribution with MCMC



What are we actually calculating with MCMC and the histogram?

$$Q(\tau_i) \approx \frac{1}{\Delta\tau_i}\int_{\beta_i} d\tau\, Q(\tau) = \frac{C}{\Delta\tau_i}\int_{\beta_i} d\tau\, \frac{Q(\tau)}{C} = \frac{C}{\Delta\tau_i}\int_0^5 d\tau\, \frac{Q(\tau)}{C}\chi_{\beta_i}(\tau)$$

$$= \frac{C}{\Delta\tau_i}\left\langle \chi_{\beta_i}(\tau)\right\rangle_{Q/C} \approx \frac{C}{\Delta\tau_i}\frac{1}{N}\sum_{i=0}^{N}\chi_{\beta_i}(\tau_i)$$

$\chi_{\beta_i}(\tau_i)$: indicator function; 1 for the bin we are interested (i), 0 everywhere else.

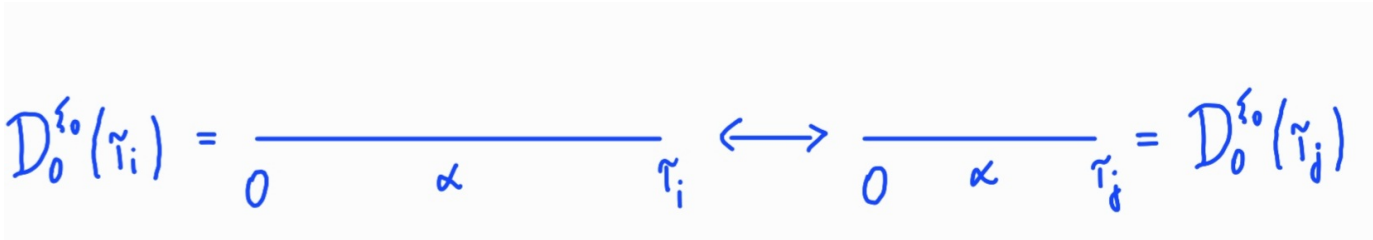
A first glimpse of DiagMC

A first glimpse of DiagMC

We can get a first taste of Diagrammatic Monte Carlo by simply reinterpreting the last exercise:

- view $Q(\tau) = e^{-\alpha\tau}$ as a Feynman diagram

- perform MCMC in the space of all diagrams: in this case, we only have 0^{th}-order diagrams
- use updates to propose a new diagram using $\mathcal{D}_0^{\xi_0}(\tau)$ as its statistical weight, i.e.



- accept the proposed diagram with probability

$$P_{acc} = \min\left\{1, \frac{\mathcal{D}_0^{\xi_0}(\tau_j)p(\tau_i|\tau_j)}{\mathcal{D}_0^{\xi_0}(\tau_i)p(\tau_j|\tau_i)}\right\}$$

General aspects of DiagMC

In general, **Diagrammatic Monte Carlo** methods let us evaluate a series:

Diagrammatic series

$$Q(\{y\}) = \sum_{n=0}^{\infty} \sum_{\xi_n} \int \cdots \int \mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)\, dx_1 \ldots dx_n$$

- function of interest $Q(\{y\})$ depending on a set of *external* variables $\{y\}$, e.g $G(\mathbf{k}, \tau)$
- n is the running index of the series and specifies the current order
- ξ_n indexes different diagrams of the same order (e.g. different topology)
- $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)$ represents a Feynman diagram
- $x_1, \ldots, x_n$ are integration variables, e.g. wave vectors and internal time points

How to evaluate such a series? $\Longrightarrow$ DiagMC

DiagMC - How does it work?

$$Q(\{y\}) = \sum_{n=0}^{\infty} \sum_{\xi_n} \int \cdots \int \mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)\, dx_1 \ldots dx_n$$

- interpret $Q(\{y\})$ as a distribution for $\{y\}$
- interpret $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)$ as a distribution for $\{\{y\}; x_1, \ldots, x_n\}$
- simulate $Q(\{y\})$ using a **Metropolis-Hastings** algorithm by sampling diagrams stochastically
- use the value of $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)$ as the statistical weight of the diagram
- collect statistics for $\{y\}$, e.g. in a histogram
- **updates** to propose new diagrams:
  - change the diagram order n
  - change the topology ξ_n
  - change integration variables $x_i, \ldots, x_n$ or change external variables $\{y\}$
- Advantage: using the value of the diagram as statistical weight allows to pick up the most important diagrams (order, topology, etc.). If you would fix the order/topology it would results in standard MCMC (DiagMC more general)

DiagMC - Updates

Class I:

- do not change the order n of the diagram (number of integration variables)
- change topology ξ_n, external variables $\{y\}$ or integration variables x_i
- suppose the current diagram is $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_i, \ldots x_n)$ and we propose a new diagram $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_i', \ldots x_n)$ from the distribution $p(x_i'|x_i)$

$$P_{acc} = \min\left\{1, \frac{\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_i', \ldots x_n) p(x_i|x_i')}{\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_i, \ldots x_n) p(x_i'|x_i)}\right\}$$

DiagMC - Updates

Class II:

- change the order n of diagrams
- suppose the current diagram is $\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n)$ and we propose a new diagram $\mathcal{D}_{n+m}^{\xi_{n+m}}(\{y\}; x_1, \ldots, x_n, \ldots x_{n+m})$ from the distribution $p(x_{n+1}, \ldots, x_{n+m})$ using update $\mathcal{A}$ ($\mathcal{B}$ is the inverse of $\mathcal{A}$, *detailed balance*)

$$P_{acc} = \min\left\{1, \frac{p_{\mathcal{B}}}{p_{\mathcal{A}}} \frac{\mathcal{D}_{n+m}^{\xi_{n+m}}(\{y\}; x_1, \ldots, x_n, \ldots x_{n+m})}{\mathcal{D}_n^{\xi_n}(\{y\}; x_1, \ldots, x_n) p(x_{n+1}, \ldots, x_{n+m})}\right\}$$

$\frac{p_{\mathcal{B}}}{p_{\mathcal{A}}}$: so-called context factor, it controls how updates are proposed and done. Necessary to guarantee detailed balance.

- in Class II variables are not updates: no need for term $p(x_i|x_i')/p(x_i'|x_i)$

DiagMC - Workflow

Input: initial diagram $\mathcal{D}^{(0)} \leftarrow (\{y^{(0)}\}; x_1^{(0)}, \ldots, x_n^{(0)}, n^{(0)}, \xi_n^{(0)})$,
update procedures $\{U_1, \ldots, U_k\}$,
update probabilities $\{p(U_1), \ldots, p(U_k)\}$;
Output: histogram of $Q(\{y\})$;

initialize histogram[];
initialize diagram $\mathcal{D}_{cur} \leftarrow \mathcal{D}^{(0)}$;
while not converged **do**
  choose an update U_i from $\{U_1, \ldots, U_k\}$ with probability $p(U_i)$;
  propose a new diagram $\mathcal{D}_{new} \leftarrow (\{y'\}; x'_1, \ldots, x'_{n'}, n', \xi'_{n'})$ according to U_i;
  calculate acceptance ratio R;
  draw random uniform number r;
  if $R \geq r$ **then**
    accept the proposed diagram: $\mathcal{D}_{cur} \leftarrow \mathcal{D}_{new}$;
  else
    reject the proposed diagram: $\mathcal{D}_{cur} \leftarrow \mathcal{D}_{cur}$;
  end if
  histogram[{y}]$\leftarrow$ histogram[{y}]+1;
end while
return histogram;